

简单机器学习方法

Simple Machine Learning Algorithms

明玉瑞 Yurui Ming
yrming@gmail.com

声明

Disclaimer

- ▶ 本讲义在准备过程中由于时间所限，所用材料来源并未规范标示引用来源。所引材料仅用于教学所用，作者无意侵犯原著者之知识产权，所引材料之知识产权均归原著者所有；若原著者介意之，请联系作者更正及删除。

These slides have been developed free-of-charge for educational purposes only and without any commercial interest. However, time limit during the preparation incurs the situation that not all the sources of the materials (texts or images) are properly referenced or clearly manifested. But all contents are solely for teaching and the author is with no intention to infringe the copyright bestowed on the originators and all credits go to corresponding IP holders. Please address the author have you had any concern so he can act on a remedy including deletion per request.

实例学习

Instance-based Learning

- ▶ 实例学习或基于实例的学习是最简单的机器学习算法之一，这类算法也被称作惰性学习算法。实例学习并不会根据训练集构建学习模型，而是将模型的计算过程推迟至接收到新的测试样本（查询点）时才进行。

Instance learning is one of the simplest machine learning algorithms. Such methods are also called lazy learning algorithms. Because they do NOT compute a model when they are given a training set but postpone the computation of the model until they are given a new test instance (query point).

- ▶ 实例学习的核心思想是彼此邻近或特征相似的两个样本，应当被赋予相同的标签。当给定测试集中的一个输入样本时，需要找出训练集中与其相似的样本，根据训练集中找出的样本的性质，推断输入样本的性质，例如标签等。

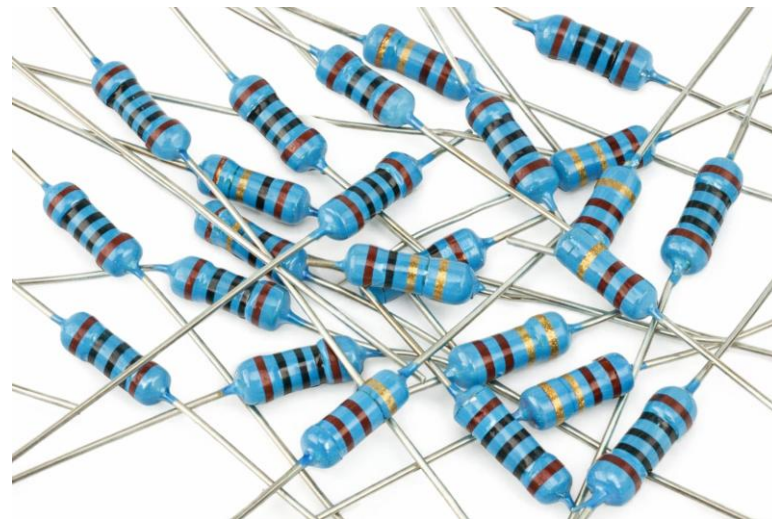
Fundamental philosophy of instance learning assumes that two instances that are close to each other or similar to each other they should share with the same label. Given an input in the test set, similar ones should be found from the training set. And based on the samples from the training set, properties of the input sample can be deduced, for example, the label.

例子

Example

- 常用的分立电阻的阻值是由电阻上打印的色环来标识的。每个色环代表特定的阻值，所有色环代表的阻值加起来，便是整个电阻的阻值。记住并快速计算电阻的阻值并不是一件轻松的事情。当因为某种原因将许多电阻混在一起的时候，有效的分拣方法便是基于实例学习的思想，每个类别挑出一个，用剩下的去比较，色环相同的归为一类。

The resistance values of commonly used discrete resistors are indicated by the color bands printed on them. Each color band represents a specific value, and taken together the bands determine the resistor's overall resistance. Remembering and quickly calculating resistor values is not an easy task. When many resistors are mixed together for some reason, an effective way to sort them is based on the idea of example-based learning: select one representative from each category, then compare the remaining resistors with it, grouping those with identical color bands into the same category.



k 近邻分类器

k -Nearest Neighbors Classifier

- ▶ 实例学习有时又称基于记忆的学习，因为其核心原理是将训练样本存储在查询表中，并基于这些已存储的样本进行插值计算。因此，该方法的空间复杂度为一般为 $O(N)$ 。

Instance-based Learning is also known as memory-based learning since what they do is store the training instances in a lookup table and interpolate from these. The memory consumption for these algorithms is generally $O(N)$.

- ▶ 实例学习的一个典型算法是 k 近邻分类器，即给定一个查询样本 x^0 ，找出与 x^0 距离最近的 k 个训练样本 $x^{(i)}$ （其中 $i = 1, 2, \dots, k$ ），随后通过这 k 个近邻的多数投票，完成对查询样本的分类。为了避免出现投票平票的情况，我们通常选择奇数作为 k 的取值。

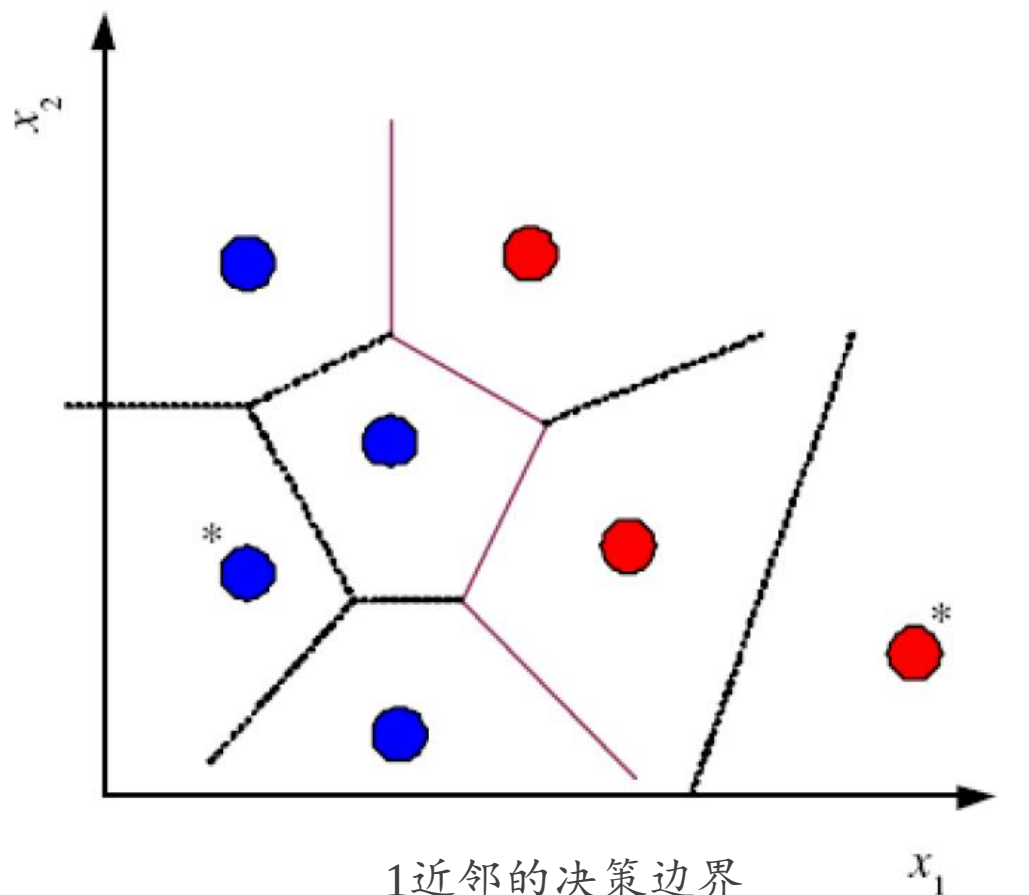
One typical algorithm of instance-based learning is the k -Nearest Neighbors Classifier. Given a query point x^0 , we find the k training points $x^{(i)}$ ($i = 1, 2, \dots, k$), Then classify using majority vote among these k neighbors. Notably, to avoid the tie, we usually choose k as an odd number.

k 近邻分类器

k -Nearest Neighbors Classifier

- 如果样本所有属性（特征）均为连续数值型，可使用欧氏距离，例如 $d(x, x^0) = \|x - x^0\|_2$ ；若属性值为离散型，则可使用汉明距离，通过统计属性不匹配的数量来计算： $d(x, x^0) = \sum_{i=1}^n \mathbf{1}(x \neq x^0)$ 。

If all attributes (features) are real-valued, we can use Euclidean distance, that is $d(x, x^0) = \|x - x^0\|_2$; If the attribute values are discrete, we can use Hamming distance, which counts the number of nonmatching attributes, aka., $d(x, x^0) = \sum_{i=1}^n \mathbf{1}(x \neq x^0)$.



1近邻的决策边界

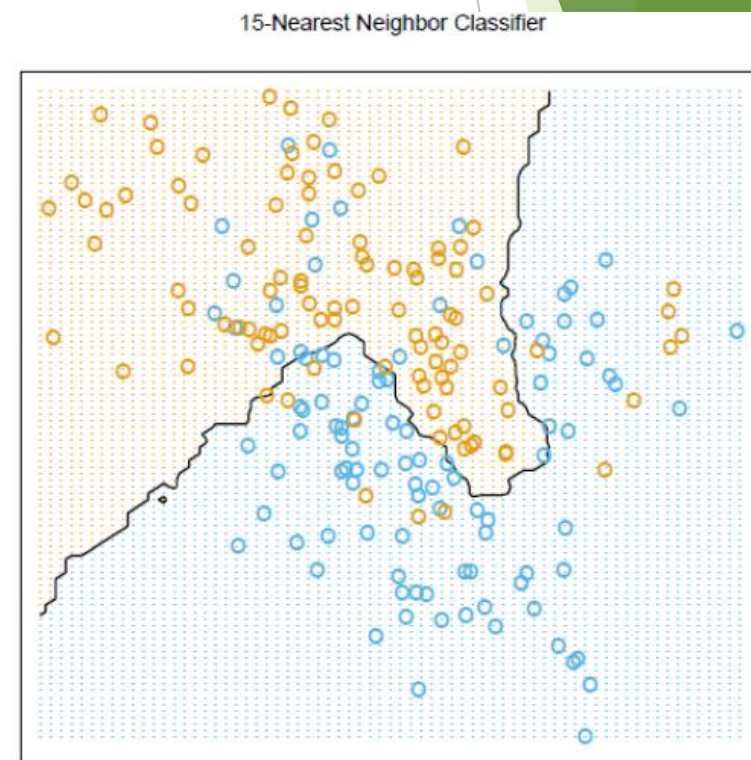
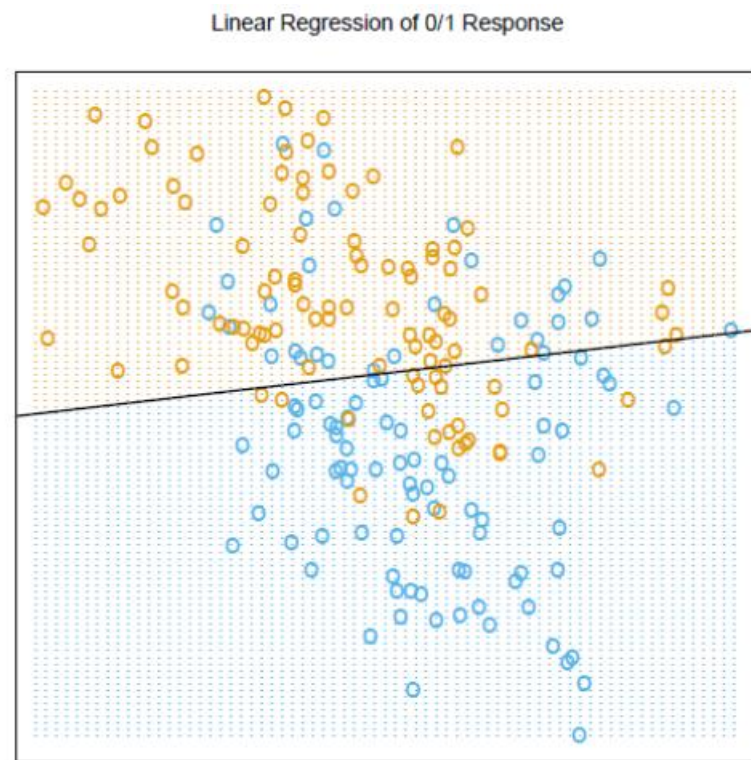
1-Nearest Neighbor Decision Boundary

k 近邻分类器

k -Nearest Neighbors Classifier

- 虽然是最简单的机器学习算法，在某些情况下， k 近邻分类器的性能要好于其他需要建立模型的算法。

Being the simplest machine learning algorithm, k -Nearest Neighbors Classifier can out-perform other algorithms requiring setup model explicitly.



k 近邻分类器

k -Nearest Neighbors Classifier

- 最原始的 k 近邻算法，与训练集中的所有样本数量成正比，当样本数量非常大时，由于需要计算与所有样本的距离，再取前 k 个，会比较耗时。一种思路是所谓的压缩近邻算法，即不必与所有训练集合中的点做比较，先找中具有代表性的点，对于测试点，仅计算与这些代表点的距离。

The original k -Nearest Neighbor algorithm has a time cost proportional to the number of samples in the training set. When the sample size is extremely large, it is rather time-consuming, since distances to all training samples must be calculated first before selecting the top k nearest ones. One solution is the so-called Condensed Nearest Neighbor algorithm. Instead of comparing with every point in the training set, this algorithm first identifies representative points. For a test point, only the distances to these representative points need to be computed.

$Z \leftarrow \{x^i\}$ (randomly assignment)

Repeat

For all $x \in \mathcal{X}$ (in random order)

Find $x' \in \mathcal{X}$ s.t. $\|x - x'\| = \min_{x^j \in Z} \|x - x^j\|$

If $\text{class}(x) \neq \text{class}(x')$ add x to Z

Until Z does not change

k 近邻分类器

k -Nearest Neighbors Classifier

- 在 k 近邻算法中，采用不同的距离度量方式，会得到截然不同的结果，因此，我们计算距离时需格外注意。一个好的做法是对不同属性的量纲，做归一化处理，使其均值为0、方差为1： $\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j}$ 。

Using different distance measurements will give very different results in k-NN algorithm, hence, we need to be careful when compute the distance. A practice is to normalize the scale between different attributes to have mean zero and variance 1: $\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j}$.

- 在实际应用中，距离函数 $d(x^i, x^j)$ 的选取并非随意之举。所选的距离函数需满足一个核心要求：若样本 x^i 与 x^j 属于同一类别，则二者的距离应尽可能小；若二者分属不同类别，则距离应尽可能大。

In practice, Finding a distance function $d(x^i, x^j)$ is not a arbitrary choice. the candidate should guarantee that if x^i and x^j are belong to the class the distance is small and if they are belong to the different classes the distance is large.

二分类问题

Binary Classification Problem

- ▶ 二分类问题是监督学习中的一种基本任务，其目标是将给定的输入样本判定为两个可能类别之一。具体做法是通过已标注的训练样本，用选定的模型或算法构建一个判别函数，通过学习将输入特征映射为二值标签（如0/1或-1/+1），从而对未见样本做出正确判断。

A binary classification problem is a supervised learning task in which the objective is to assign each input sample to one of two possible classes. Given a set of labeled examples, the learner aims to construct a decision function that maps input features to a binary label (such as 0/1 or -1/+1), and can correctly classify unseen data.

- ▶ 用数学语言描述为下：对于给定训练集 $S = \{(x^i, y_i)\}_{i=1}^l \subset \mathbb{R}^n \times \{-1, 1\}$ ，其中样本 $x^i \in A_+ \Leftrightarrow y_i = 1$ ， $x^i \in A_- \Leftrightarrow y_i = -1$ 。寻找函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ，通过基于 S 按规则 $f(x) \geq 0 \Rightarrow x \in A_+$ 且 $f(x) < 0 \Rightarrow x \in A_-$ ，从而能够预测未知新的数据的标签。

Expressed in math notation, given a training dataset $S = \{(x^i, y_i)\}_{i=1}^l \subset \mathbb{R}^n \times \{-1, 1\}$, where $x^i \in A_+ \Leftrightarrow y_i = 1$ and $x^i \in A_- \Leftrightarrow y_i = -1$. Find a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ by learning from data $f(x) \geq 0 \Rightarrow x \in A_+ \wedge f(x) < 0 \Rightarrow x \in A_-$, to predict the unseen class label for new data.

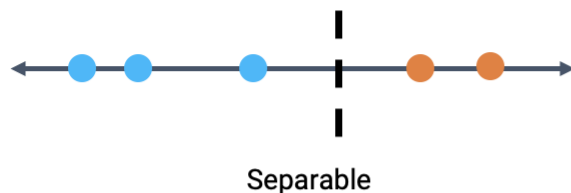
二分类问题

Binary Classification Problem

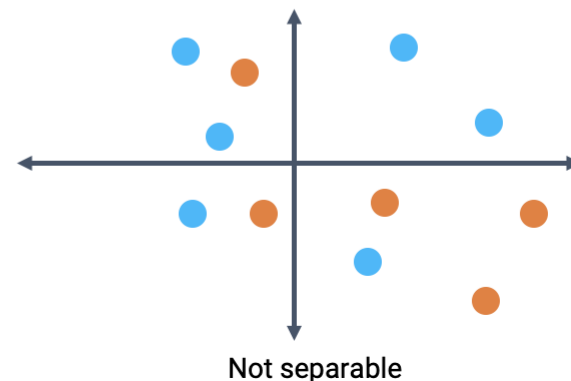
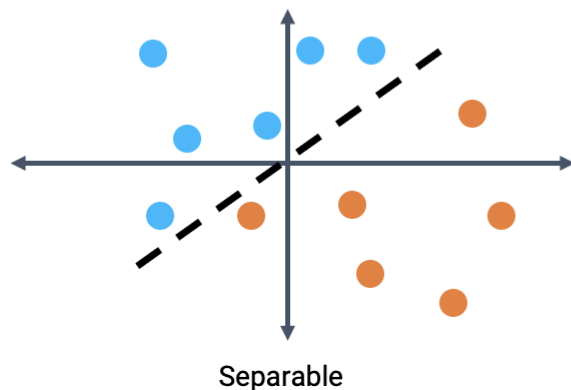
- 二分类问题通常用于“是/否”、“对/错”、“正/负”等决策场景。如果问题对应的数据集本身是线性可分的，其对应于线性模型的形式： $f(x) = w^T x + b$ 。

Binary classification is commonly used in problems involving yes-no, true-false, or positive-negative decisions. If the dataset corresponding to the problem is linearly separable, then the model has the simplest linear form: $f(x) = w^T x + b$.

Dataset with one feature:



Dataset with two features:

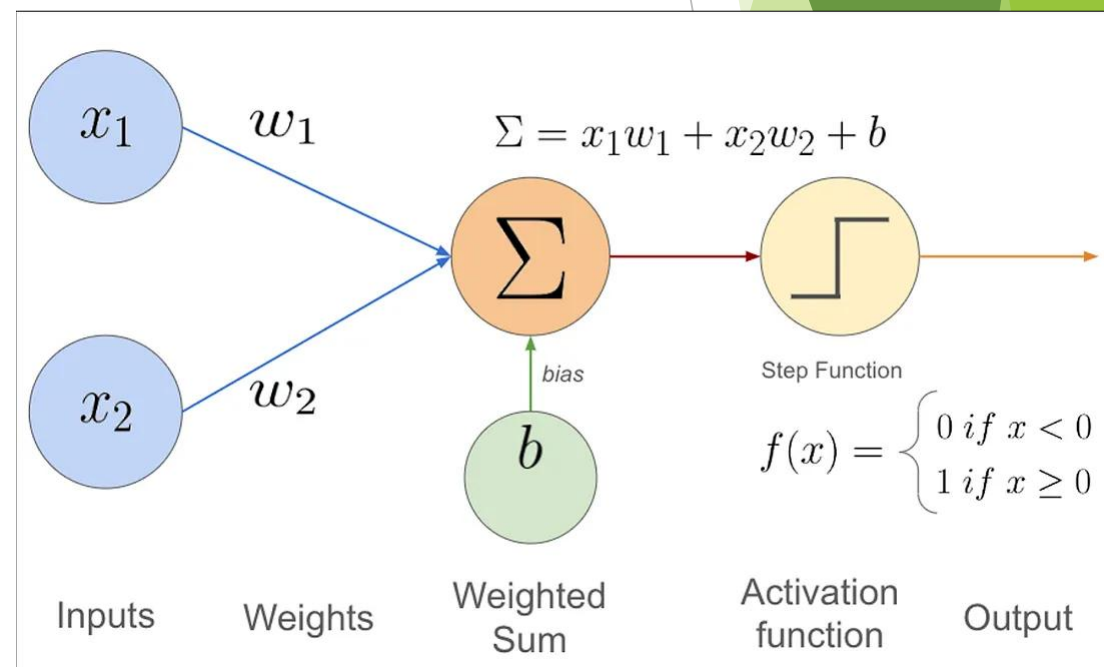


感知机算法

Perceptron Algorithm

- 用于线性可分问题的算法或模型有多种，如线性判别器，感知机等，我们这里介绍感知机。其是由Frank Rosenblatt于1957–1958年在美国的康奈尔航空实验室提出的。该计算模型受大脑神经元启发，通过基于错误的迭代学习过程从数据中学习，用于实现二分类。最初的设计动机是探索一种能够模仿人脑简单神经元行为的机器学习和模式识别方法。

There are several algorithms or models can be applied to linearly-separable problem, for example, Linear Discriminant Analysis, Perceptron, etc. Here we introduce perceptron, which was invented by Frank Rosenblatt in 1957–1958 at the Cornell Aeronautical Laboratory in the United States. It was designed as a computational model inspired by biological neurons, intended to perform binary classification and learn from data through an error-driven iterative process. The original motivation was to explore machine learning and pattern recognition in a way that mimicked the human brain's simple neuron behavior.



感知机算法

Perceptron Algorithm

- ▶ 基于训练集，感知机的学习方法如右图伪代码所示。该学习方法称为错误驱动的在线学习方法：

Based on the data from training set, the learning process for perceptron is as in the right. This learning algorithm is an on-line and mistake-driven approach:

$$R = \max_{1 \leq i \leq l} \|x^i\|$$

Repeat

for $i = 1$ to l

if $y_i \cdot (\langle w^k, x^i \rangle + b_k) \leq 0$ then

$$w^{k+1} \leftarrow w^k + \eta y_i \cdot x^i$$

$$b_{k+1} \leftarrow b_k + \eta y_i R^2$$

$$k \leftarrow k + 1$$

end if

until no mistakes made within the for loop

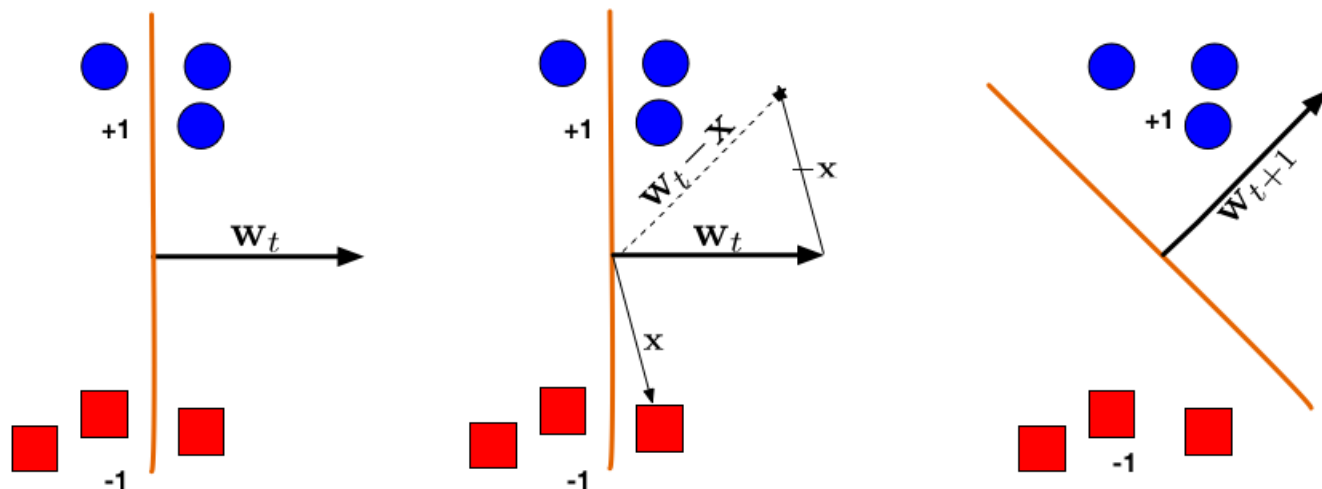
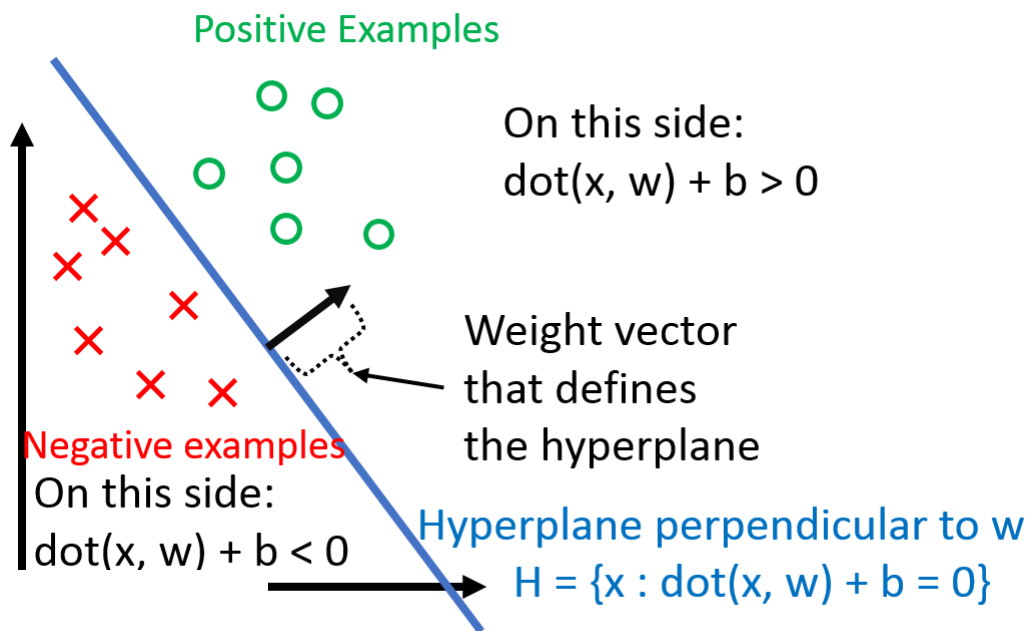
return: $k; (w^k, b_k)$

感知机算法

Perceptron Algorithm

- 其学习方式的动机与学习过程的图示如下所示:

The motivation for learning and the adjusting process are illustrated as in the following:



感知机算法

Perceptron Algorithm

- 下面来说明感知机算法的正确性，即在迭代过程中 $y_i \cdot (\langle w^{k+1}, x^i \rangle + b_{k+1}) > y_i \cdot (\langle w^k, x^i \rangle + b_k)$ 。

Now we show the efficacy of the perceptron learning algorithm, aka., during the iterative process, $y_i \cdot (\langle w^{k+1}, x^i \rangle + b_{k+1}) > y_i \cdot (\langle w^k, x^i \rangle + b_k)$.

$$\begin{aligned} & y_i \cdot (\langle w^{k+1}, x^i \rangle + b_{k+1}) \\ &= y_i \cdot (\langle w^k + \eta y_i \cdot x^i, x^i \rangle + b_k + \eta y_i R^2) \\ &= y_i \cdot (\langle w^k, x^i \rangle + \eta y_i \cdot \langle x^i, x^i \rangle + b_k + \eta y_i R^2) \\ &= y_i \cdot (\langle w^k, x^i \rangle + b_k + \eta y_i \cdot \langle x^i, x^i \rangle + \eta y_i R^2) \\ &= y_i \cdot (\langle w^k, x^i \rangle + b_k) + \eta y_i^2 \cdot (\langle x^i, x^i \rangle + R^2) \end{aligned}$$

- 显然 $\eta y_i^2 \cdot (\langle x^i, x^i \rangle + R^2) > 0$ ，即 $y_i \cdot (\langle w^{k+1}, x^i \rangle + b_{k+1}) > y_i \cdot (\langle w^k, x^i \rangle + b_k)$ 。

It is obvious that $\eta y_i^2 \cdot (\langle x^i, x^i \rangle + R^2) > 0$, indicating $y_i \cdot (\langle w^{k+1}, x^i \rangle + b_{k+1}) > y_i \cdot (\langle w^k, x^i \rangle + b_k)$.

感知机算法

Perceptron Algorithm

- 实际上，感知机模型与学习方法在当时引起了巨大的轰动，特别是在计算机逐步推广应用的背景下，激发了人们构建数字大脑的乐观情绪。纽约客曾对此进行了专题报道。

In fact, the perceptron model and its learning method triggered a huge excitement at the time. Especially along the gradual adoption of computers, it sparked widespread optimism about the possibility of building a digital brain. The New Yorker even published an article about it.,

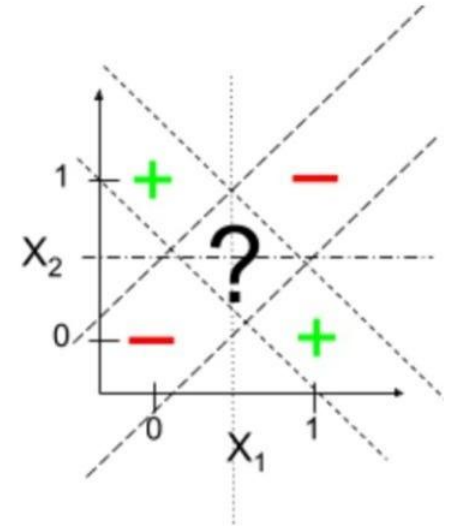
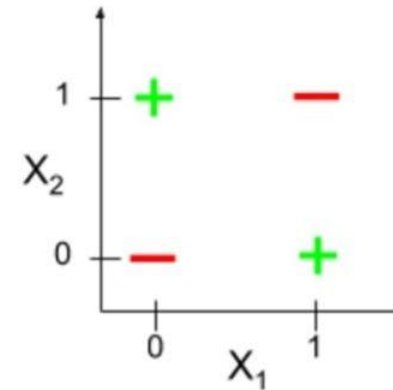


感知机算法

Perceptron Algorithm

- 但是，囿于各种原因，后来人工智能的发展并不如预期，特别地，明斯基于1969所提出的XOR问题，一个著名的简单非线性可分数据集，揭示了感知机的缺陷，这一问题促成了“人工智能寒冬”的到来。

However, due to various reasons, the advancement of Artificial Intelligence was not as expected. Especially, Minsky proposed the XOR problem in 1969, a famous example of a simple non-linearly separable data set, which contributed to the A.I. Winter.



Deep Forward Network

XOR PROBLEM

感知机算法

Perceptron Algorithm

- ▶ 实际上，感知机算法与Hebb法则有很深的渊源。在神经网络的模型建立后，并没有意识到神经元的可塑性的重要意义，因此，当时一段时间内，人工神经网络的权重是固定的。因此神经网络的应用有很大的局限。当意识到可塑性的意义时，却发现缺乏一种有效的学习权重的方法。后来，神经学家Hebb根据Pavlov的实验，提出了Hebb法则，其演化成了Frank Rosenblatt博士的感知机与其学习方法。尽管当时已经提出了神经网络的现代的一些概念，如隐含层，在这个视角下，XOR问题根本就不是问题。但由于缺乏全局的学习方法，带隐含层的神经网络并不能有效地学习，从而解决这类问题。这一直要到80代中期，直到反向传播算法的提出，神经网络才能重装上阵。

In fact, the perceptron learning algorithm has deep roots in Hebb's rule. After early neural network models were proposed, the importance of neuronal plasticity was not fully recognized. As a result, for a period of time, the weights in artificial neural networks were considered fixed, which severely limited their applicability. When the significance of plasticity was eventually acknowledged, it became apparent that there was a lack of effective methods for learning and adjusting weights. Later, based on Pavlov's experiments, the neuroscientist Donald Hebb proposed what is now known as Hebb's rule. This rule subsequently evolved into the perceptron model and its learning algorithm developed by Dr. Frank Rosenblatt. Although some modern concepts of neural networks—such as hidden layers—had already been proposed at that time, from this perspective the XOR problem was fundamentally not an issue. However, due to the absence of a global learning mechanism, neural networks with hidden layers were unable to learn effectively and thus could not solve such problems. It was not until the mid-1980s, with the introduction of the backpropagation algorithm, that neural networks were able to re-emerge and regain prominence.